



Oranjewood Assignment



Approach

Consultant-friendly explanations backed by technical architecture and implementation detail.

Working style

Default Odoo first, safe automation second, module implementation & packaging when stable.

Kevin Kahumba

Odoo Developer

Clean architecture, practical automation and Odoo solutions shaped by real project delivery.

- 1 **Current role**
Odoo Developer at Therp B.V.
- 2 **Experience**
Working with Odoo from v8 through v19.0.
- 3 **Core stack**
Python, TypeScript, JavaScript, OWL, CSS and XML.
- 4 **Community**
OCA member and contributor to open-source Odoo initiatives.
- 5 **Selected work**
BMair International, Inuka S.A., Human Company and Bryntum-related implementations.
- 6 **Focus**
Odoo ERP functional workflows, automation, development, tooling, OWL framework, API integrations, migrations and A.I integration and application.

Works at Therp Bv experience in Odoo v8 – v19.0, is an OCA member.

Certification & Current Interests

- Last year I participated in an [OWL training](#) course that was offered through OCA and trained by members of [ForgeFlow](#). After the training I got the certification shown on the right side ->
- Odoo certifications for v17 - v19.
- I recently worked on migrating OCA from v14 to v18.0
- I'm working on a module to visualize stock, mrp and its operations using OWL and [three.js](#) library.
- Learning to work with [Odoo.sh](#)
- Learning OCA Openupgrade migration tool.
- Learning mobile programming and using Odoo for REST API.
- Learning Odoo Integration with A.I models and agents.

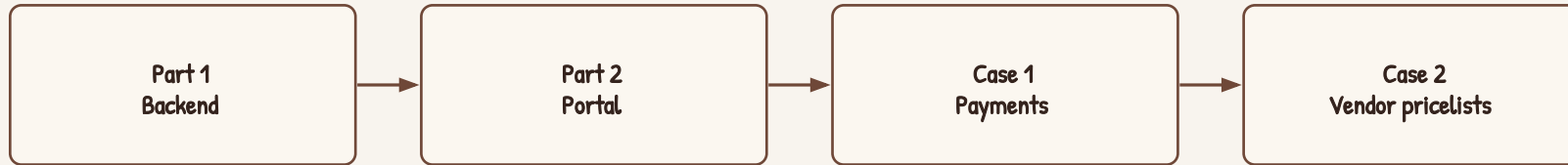




Odoo v17.0

Technical Assignment

Backend customization, portal widget, bank-transfer automation and vendor pricelist workflows.



The assignment was developed around Odoo v17 and answers target Odoo v17.0.

Assignment Breakdown

A complete answer to the Python/XML, portal, and consulting cases.

Part 1 - Backend

- Add sale.order margin fields
- computed manager approval flag
- form view section and red alert
- button/server action with chatter update

Part 2 - Portal

- Portal order detail button
- JSON controller route
- publicWidget and ajax.jsonRpc
- Odoo-consistent inline popup

Case 1 - Payments

- Default bank transfer flow
- statement sync/import path
- custom payment provider option
- 10x volume risk and mitigation

Case 2 - Vendors

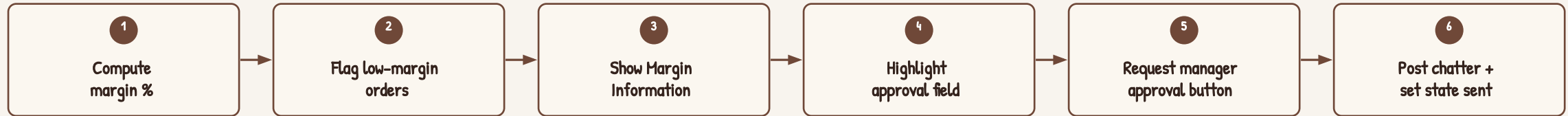
- 45K products
- 25 vendors
- 25% assortment sold
- server actions + cron

Assignment breakdown and summary approach to each question

Part One

Part 1: Backend – Python Customization

Extend sale.order with margin intelligence and a manager approval path.



Data additions

- margin_percentage: Float, 2 decimals
- requires_manager_approval: Boolean
- computed from margin percentage

View additions

- new section: Margin Information
- show both fields on sale order form
- style low-margin approval warning in red

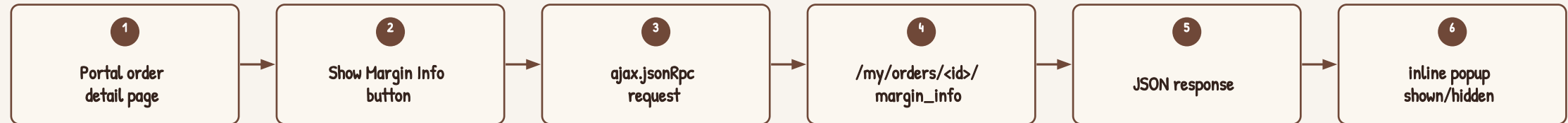
Button behavior

- post manager approval request in chatter
- move quotation to sent when needed
- keep logic simple and auditable

Question 1 Part 1 Backend workflow

Part 2: Frontend – Portal Customization

Add a small interactive margin panel to the portal sale order detail page.



Controller route

- auth=user / portal-safe access check
- return margin_percentage and approval flag
- Never expose orders the portal user cannot access

QWeb placement

- Extend portal sale order template
- Place button near Print / Download Invoice
- Reuse Bootstrap/Odoo portal classes

JS behavior

- publicWidget registry
- ajax.jsonRpc fetch
- fade or slide inline popup

Question 1 Part 2 Frontend Workflow

Case One

Case 1: What the consultant is asking

B2C tile ecommerce on Odoo.sh, peak sales, 17% bank transfer, limited accounting capacity.

Business situation

- Website sales are 3-4x higher in peak season
- Bank transfer used by about 17% of orders
- Order confirmed only after payment
- Sales team cannot touch bank statements

Operational bottleneck

- Bookkeeper works part-time
- Bank statements are synced automatically
- Manual reconciliation delays delivery
- Absence or sickness creates single-point risk

Questions to answer

- Default Odoo flow
- Customization needed or not
- 10x volume impact
- things to avoid
- Estimate for answer and delivery

25-50 orders/day

baseline volume before 10x scenario

The answer must be clear for the consultant and technically safe for the project lead.

Case 1: Default Odoo Customer Payment Flow

How default Odoo v17.0 Enterprise handles the bank-transfer checkout journey.



Frontend

- Customer logs in and adds items to cart.
- At checkout Odoo shows company bank details.
- Reference and amount guide the customer payment.
- Customer waits for confirmation before delivery.

Backend

- Company configures bank transfer instructions.
- Accounting tracks statements and matches payment references.
- If fully paid, invoice and sales flow can continue.
- Partial payments require follow-up and exception handling.

Default Odoo works well when references are clean and accounting follows a controlled reconciliation flow.

Case 1: Default features to use first

Avoid rebuilding what Odoo already gives you.

Payment and invoicing

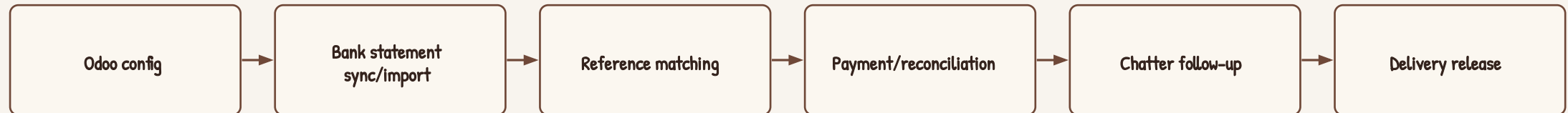
- Configure wire transfer instructions
- Invoice Online Payment supports portal payment visibility
- Auto invoice works best when invoicing policy permits it

Banking and reconciliation

- Bank synchronization or statement import
- accountant keeps final financial control
- references allow matching to sale order or invoice

Batch / statement imports

- Batch payments can help grouping
- CSV / OFX imports are useful if bank supports files
- OCA bank import modules are fallback for Community



Core idea: Import statements, match references, automate repetitive checks, escalate exceptions.

Case 1: Bank statement import automation

How server actions and cron jobs can automate the wire-transfer validation path.



Tracking Fields

- last_fetch_date
- stmt_has_been_processed
- auto_process_sale_orders

Processing Rules

- Process only new files once.
- Reuse OFX / CSV import modules first.
- Match by reference before delivery.
- Post chatter updates for visibility.

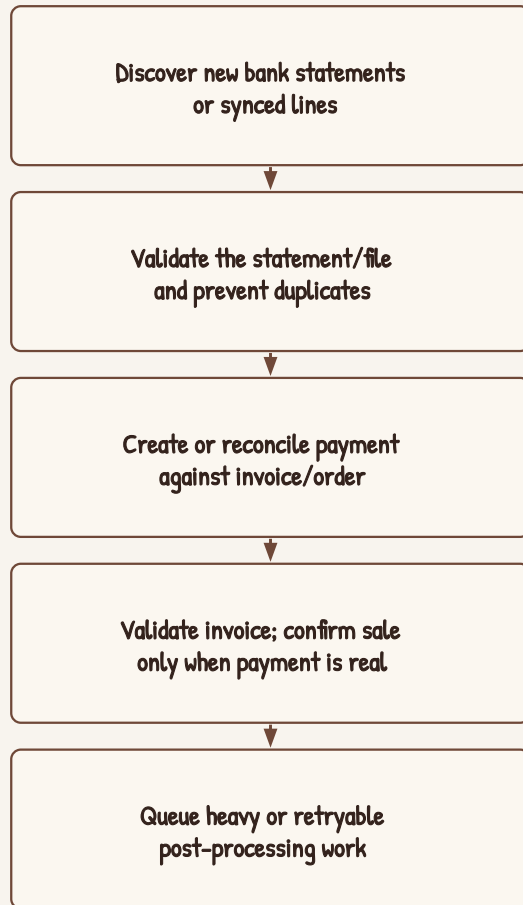
Safety Switch

- Use a config option to enable the automation.
- Disable cron when manual review is preferred.
- Queue or batch heavy work off-hours if volume grows.

Automate carefully: import, validate, reconcile, and only then release the downstream sales flow.

Case 1: Server Actions, Cron and Queue Job Design

A safe automation should separate discovery, validation, reconciliation and downstream actions.



Use queue_job for

- Large matching or reconciliation processing
- Callbacks/retries
- Invoice validation and picking follow-up
- Work that can safely retry

Do not queue

- Tiny synchronous UI checks
- Logic that must block checkout immediately
- Unsafe tasks with no idempotency key
- Work that must be manually approved first

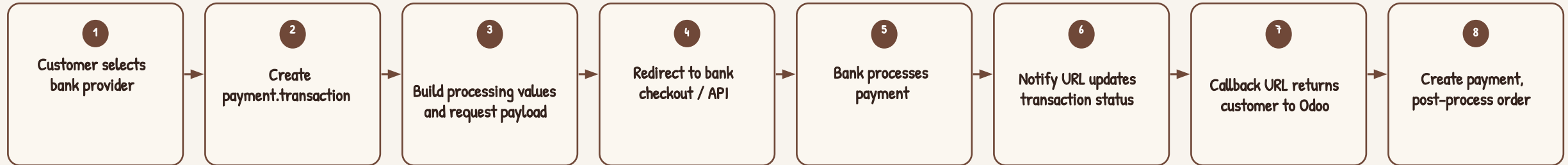
Design contract

- Every transaction/file must have a unique reference or fingerprint.
- Every step must be idempotent, logged, and safe to rerun.
- Exceptions should go to humans; routine checks should go to automation.

Queue jobs improve reliability only when the process is idempotent and monitored.

Case 1: Custom Payment Provider/API Flow

Best path when the bank exposes an API or hosted checkout for direct confirmation status.



Reuse Odoo lifecycle

- Payment provider configuration
- Payment.transaction state management
- notification handling
- Payment creation and order callback
- Error, pending and retry paths

Good queue_job candidates

- Invoice validation
- payment post-processing
- Stock / picking follow-up
- Heavy callback work and retries

Key controls

Idempotency, Secure Callbacks, Clean Reference Mapping, and Clear Monitoring.

Use the API/provider route when the bank can confirm payments directly back into Odoo.

Case 1: 10x order volume impact

The risk is not Odoo itself; it is manual bottlenecks and unsafe automation.

Main risks

- More payments to verify
- More statement lines to reconcile
- More customer reference mistakes
- Higher duplicate-processing risk
- More lock contention if automation is poorly written

Mitigation

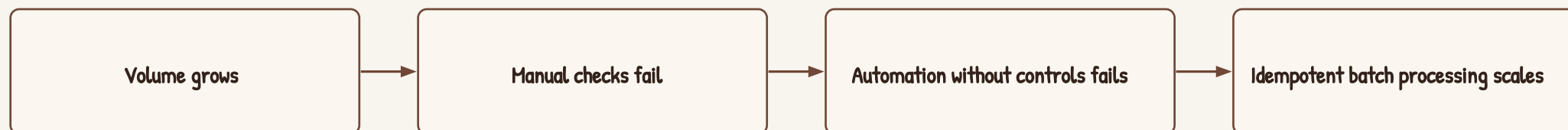
- Strict payment references
- Bank import or sync plus exception handling
- Queue/batch work outside peak hours
- Clear notifications and monitoring
- Humans handle exceptions only

Technical pressure

- Larger reconciliation domains
- Potential locks during payment posting
- Large files can overload peak hours
- Indexes/domains matter at scale

Business effect

- Delayed deliveries
- Customer complaints
- accounting backlog
- Single staff dependency



Design the automation for exception only human review, not for replacing accounting controls.

Case 1: Things to avoid

Risks for the consultant and technical risks for the project lead.

Manual dependency

- Do not rely on one accountant to check every payment.
- Manual work becomes unavailable, slow, and error-prone.

Over-customization

- Do not rebuild default payment/import/reconciliation features.
- Customize only where the bank or process requires it.

Weak references

- Avoid vague payment references.
- Require sale order or invoice reference in payment instructions.

Duplicate processing

- Never process a bank file or payment twice.
- Use unique references, file hashes, or imported statement IDs.

Premature delivery

- Do not trigger delivery just because a customer says they paid.
- Wait for statement/provider confirmation.

Single point of failure

- Avoid depending on one staff member, payment method, or fragile cron.
- Add monitoring and clear fallback paths.

Simple Rule: Never ship goods until the payment path is confirmed and traceable.

Case 1: Answer and Delivery Estimate

Consultant answer quickly; implementation depends on bank/API complexity

Phase	Time	Output
Investigate and explain problem	~1h 30m	Confirm process, bank input, current Odoo settings.
Research solution options	~1h 30m	Choose default import path or provider/API path.
Implementation	~3h	Studio fields, server action/cron, or provider module changes.
Code review and testing	~1h	Test idempotency, references, partial payments and errors.
Deployment	~40m	Deploy, enable safely, monitor first run.

Answer can be prepared in about 1h 30m; full safe delivery is phased.

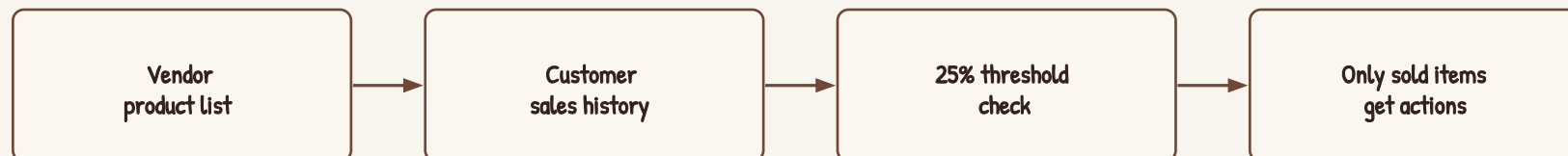
Case Two

Case 2: Vendor Pricelist Challenge

Use sales history to focus replenishment and pricelist updates.

**45K
products**

Built up over 20+ years across 25
vendors



Business aim

- Avoid asking vendors for the full catalog again.
- Update restocking/pricelist info only for products that actually sell.
- Create vendor-product focus from data instead of gut feeling.

The answer is not to automate all 45K products; it is to automate the commercially relevant 25%.

Case 2: Studio Fields

Small data model additions make the automation visible to business users.

res.partner fields

- Vendor Products Supplied - Many2Many
- Does Vendor meet 25% Sold Goods Threshold - Boolean
- Vendor Products 25% Unsold - Many2Many

product.product field

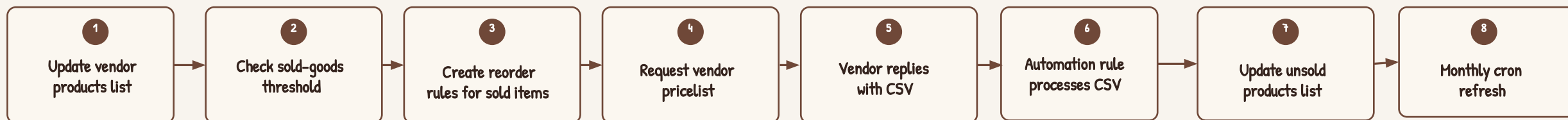
- Does product meet vendor sold threshold - Boolean
- Used to filter product variants that should not drive replenishment
- Cleared again when product becomes sold/relevant



Implementation can begin in Studio for fast 'hotpatch'; module later makes it safer for permissions, upgrades and restore points.

Case 2: Complete vendor automation workflow

From vendor catalog to focused replenishment and yearly pricelist updates.



Default Odoo Core

- Products and product.supplierinfo
- sales order lines
- Reordering rules/orderpoints
- Model mail.message and attachments

Studio/Server Actions

- Fields for vendor analysis
- Server actions for analysis and updates
- Automation rule for incoming CSV
- Cron for monthly refresh

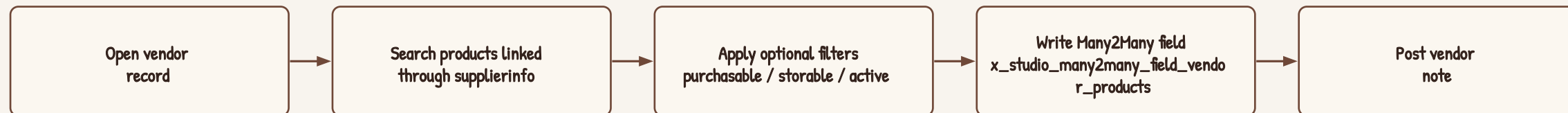
Business Control

- Manual trigger first for demo
- Purchasing reviews RFQs
- CSV validated before updates
- Chatter notes provide audit trail

This is a focused automation chain; each server action has one clear responsibility.

Server Action 1: Update vendor products list

Make the vendor record show all supplied products directly.



Purpose

- Gives a vendor-side view of supplied products.
- Works for one vendor or multiple selected vendors.
- Supports commercial partner logic.

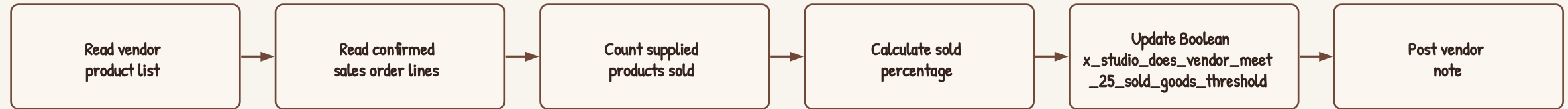
Result

- Vendor record shows supplied products.
- Easy demo of vendor-to-product relationship.
- Users can review the vendor catalog faster.

This action builds the foundation for the rest of the vendor analysis workflow.

Server Action 2: Check vendor sold goods threshold

Determine whether the vendor meets the 25% sold-goods threshold.



Inputs

- Vendor supplied products
- Confirmed sales in selected year
- Threshold rule = 25%

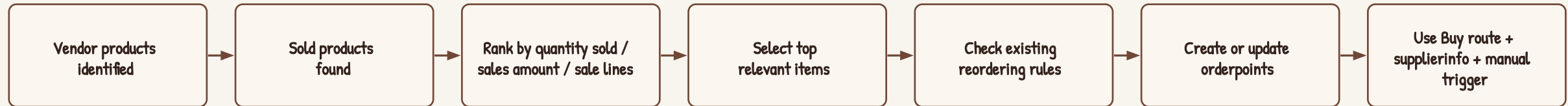
Output

- Vendor product count
- Sold product count
- Sold percentage
- Threshold passed = True / False

The threshold flag decides whether the vendor enters the focused replenishment workflow.

Server action 3: Create reordering rules for sold products

Only create replenishment logic for products that truly move.



Why manual first

- Safer for a demo
- Easy to explain to purchasing
- Lets buyers review proposals before RFQs are generated

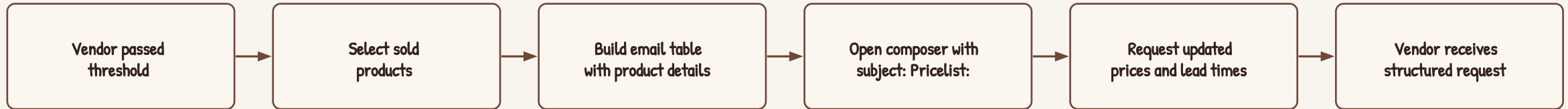
Business effect

- Vendor price comes from supplierinfo.
- Sales history decides what matters.
- Reordering rules automate future replenishment.

Replenish only what sells; avoid automating the entire vendor catalog.

Server Action 4: Request Vendor Pricelist

Generate a structured restocking/pricelist request email only for vendors that passed the threshold.



Email asks for

- Updated purchase price
- Currency
- Minimum order quantity
- packaging/box quantity
- Lead time
- Replacement or discontinued references

Why the subject token matters

- Pricelist: makes replies easy to identify.
- Supports the incoming mail automation.
- Keeps the CSV update process predictable.

Ask only for commercially relevant products instead of the vendor's full catalog.

Server Action 5: Process vendor pricelist CSV reply

Incoming vendor emails trigger a controlled CSV import path for supplierinfo updates.



CSV columns

- internal_reference
- vendor_product_code
- vendor_product_name
- price
- currency
- min_qty
- lead_time_days

Updates

- Vendor code and vendor product name
- purchase price and currency
- Minimum quantity
- Delivery lead time

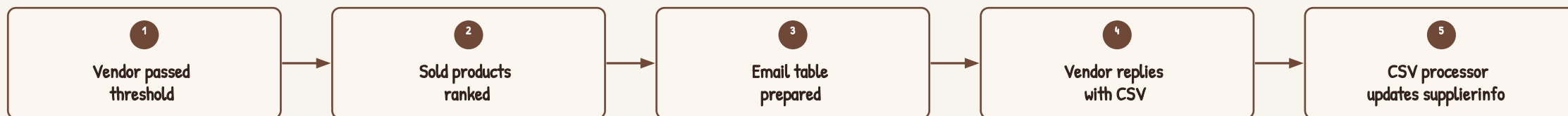
Error handling

- Invalid CSV updates nothing.
- Post a vendor note explaining the issue.
- Optional error email returns required format.

Validate first, then update supplierinfo in a predictable and auditable way.

Server Action 6: Generate restocking/pricelist email table

This is the bridge between threshold analysis and CSV-based vendor updates.



Email table includes

- Internal Reference
- Product name
- Vendor product code/name
- Quantity sold in selected year
- Current vendor price, min qty and lead time

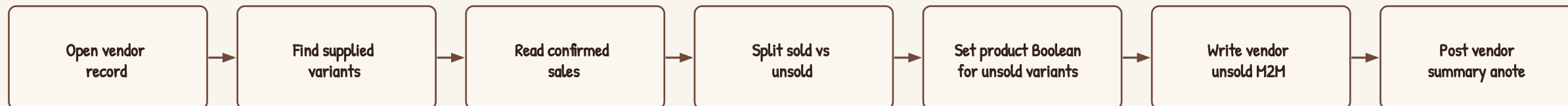
Business reason

- Avoids asking for the full catalog again.
- Gives the vendor a clear table to respond to.
- Prepares the reply for predictable CSV processing.
- Creates an auditable trail in chatter.

This action makes the vendor reply structured before automation tries to read it.

Server Action 7: Update vendor unsold products list

Flag slow or unsold product variants so they do not drive automatic replenishment.



Fields Updated

- x_studio_does_not_meet_vendor_sold_threshold_25_1
- x_studio_vendor_products_25_unsold_ids
- Clears boolean again when product is sold

Why it matters

- Shows products that are not moving.
- Avoids reorder rules for dead stock.
- Supports clean product filtering.

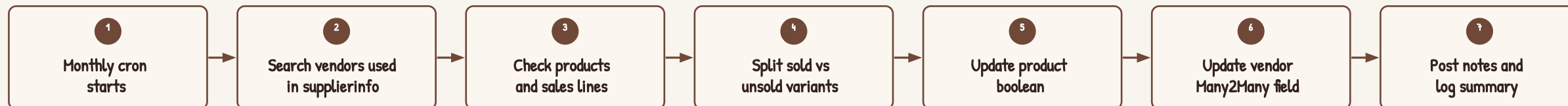
Output

- Vendor note with counts and percentages
- Product variants flagged for review
- Vendor record shows unsold list

The automation separates the vendor catalog from the active replenishment assortment.

Cron 8: Monthly unsold-product threshold refresh

Keep unsold-product analysis current for purchasing and replenishment control.



Why it matters

- Keeps slow products visible.
- Avoids replenishing products that do not move.
- Supports cleaner filtering and reporting.
- Separates active assortment from dead stock.

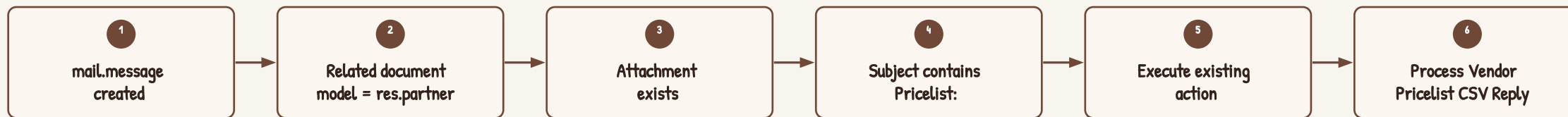
Frequency and controls

- Suggested frequency: monthly
- Can process vendors in batches
- Can limit to purchasable/storable/active products
- Supports manual review through chatter notes

Use cron to keep vendor-product intelligence fresh without manual vendor-by-vendor review.

Automation Rule: Auto-handle vendor email with CSV

The automation rule watches mail.message and delegates real work to the CSV server action.



Trigger

- After creation
- incoming email creates a new message
- avoid On save to prevent reruns

Filter

- related model is res.partner
- attachment exists
- subject contains Pricelist:

Action

- Execute existing actions
- child action processes CSV
- failure posts feedback and changes nothing

The automation rule should identify candidate emails only; validation belongs in the CSV processor.

Case 2: Controls, Performance and Governance

The solution is manageable at 45K products when it is filtered, logged and permissioned.

Avoid

- product duplications
- misconfigured vendor lines
- CSV files updating without validation
- vendor self-service portal before data controls are ready

Maintain

- access groups for who can run actions
- logging and chatter notes for traceability
- OCA modules before custom development
- audit module/logging for sensitive changes

Performance position

- 45K products is manageable when only relevant products drive daily automation
- vendor updates happen yearly or in batches
- threshold analysis can run monthly/cron and be batched

Risk is not product count alone; it is poor data, flawed matching or unsafe bulk updates.

Case 2: Implementation estimate

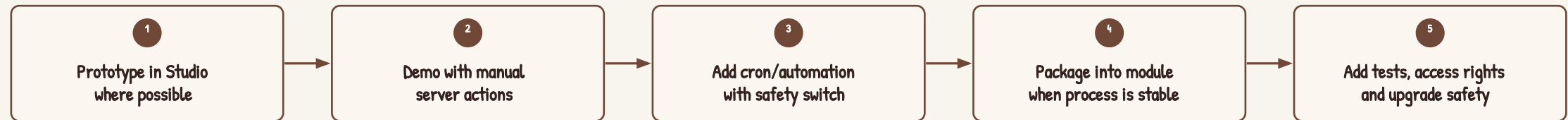
Default Odoo + Studio can handle the process; a module makes it durable.

Phase	Time	Output
Investigate and research	~1h 30m	Confirm products, vendors, fields and data quality.
Architecture and tasks	~1h 30m	Map server actions, automation rule and cron.
Implementation	~2h	Studio fields, server actions, automation rule and cron.
Testing and code review	~1h 30m	Test vendor/product links and CSV success/failure cases.
Deployment	~40m	Deploy, train users and monitor first processing pass.

All timings can reduce with AI assistance, but data validation and testing should not be skipped.

Studio first, module when stable

A practical delivery strategy for both assignment cases.



Studio is useful for

- Fields
- Simple visibility changes
- Demo-ready server action logic
- Fast consultant validation

Module is better for

- Version control and review
- Security/access rights
- Tests and migrations
- Stable reuse across databases

Do not skip

- Idempotency checks
- Clear monitoring and chatter logs
- Manual override path
- Bad-data handling

Use Studio to prove the workflow; use a module to make it maintainable.

Final Recommendation

Use the least custom solution that is safe, auditable and easy to explain.

Case 1

- Start with Odoo Enterprise payment/accounting features and bank statement imports.
- Use automated statement checks only after references and idempotency are clear.
- Use payment provider integration only when the bank/API gives reliable direct confirmation.

Case 2

- Use Studio fields for demo and visibility.
- Server actions handle vendor/product threshold logic.
- Incoming CSV automation updates vendor pricelist lines.
- Move to a custom module for persistence and upgrade safety.

The End!

Default first. Automate carefully. Package later when the flow is proven.

Thank You!

Looking forward to hearing from you.

Kind Regards.

Email: kahumba.kev@gmail.com

Mobile: [\(+254\)707-878-884](tel:(+254)707-878-884)